

Review of TRENTOS-M Cryptographic module

This page is the description of the current vulnerabilities / bugs present in the cryptographic components of TRENTOS-M in order of priority:

- In `mbed_tls/library/aes.c`
If no hardware acceleration is present (`aes_ni`), then the `mbed_tls` library uses a table look up implementation of AES. Those implementations are known to be vulnerable to cache-timing attack since 2005 (<https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>) and should not be used in real world deployment. The Sabre board for example is not having `aes_ni` and is falling back to the table look up implementation. We have to check if those instructions are also properly used by TRENTOS
When they are available (e.g. on the Raspberry Pi 3b +)
We propose to replace the unsecured AES table lookup based implementation currently used (at least on the Sabre board) by the implementation described in <https://eprint.iacr.org/2020/1123.pdf> as the code is available and (hopefully) requires little adaptation to our framework.
 [SEOS-2363 - Authenticate](#) to see issue details  [SEOS-2365 - Authenticate](#) to see issue details
- MD5 is enabled but is insecure and should be removed from the OS_Crypto by default.
 [SEOS-876 - Authenticate](#) to see issue details
- In `OS_Tls.c`
The whole TLS protocol is not functional when used with the cipher suite [OS_Tls_CIPHERSUITE_DHE_RSA_WITH_AES_128_GCM_SHA256](#). This is due to the fact that MBEDTLS does not support TLS with DH but only with ECDH. A major modification to MBEDTLS would be needed to support DH.
 [SEOS-2253 - Authenticate](#) to see issue details
- In `CryptoLibKey.c`, `generate_DHPParams` should not be used as generating safe prime of cryptographic size is a computationally very expensive and can take literally hours.
- In `CryptoLibKey.c` l.160.
The private key X is generated to have the same number of bytes than the public parameter.
If $X > P$, then X is shifted by one until $X < P$. This creates a bias in the random distribution of the private key (of which the form depends on P) and reduces the security.

Fix: sample-and-reject X with the same bit-size than P
 [SEOS-2141 - Authenticate](#) to see issue details
- `CryptoLibKey.c` L.89
The for loop does not check the condition that g is not -1 or 1 mod P nor that g is a quadratic residue mod P.
This is due to the fact that the method `mbedtls_mpi_exp_mod(&T, ..., &T, ..., ...)`; should not be used with the same mpi in input and output.

Fix: Check if $p = 1$ or $7 \bmod 8$ as g is fixed to 2.

See:
[<https://securitypitfalls.wordpress.com/2017/05/05/safe-primes-in-diffie-hellman/>
] https://proofwiki.org/wiki/Second_Supplement_to_Law_of_Quadratic_Reciprocity
 [SEOS-2142 - Authenticate](#) to see issue details
- Another concern was that `mbed_tls` primality check might return an error as previous versions of OpenTLS as described in <https://eprint.iacr.org/2019/032.pdf>.
The concern was that the number of rounds in Miller Rabin method would not be high enough. In `mbed_tls`, the number of rounds is either 40 or 50 which is giving enough assurance.